
ChessLLM: LLM Chess Engine with Reinforcement Learning

Jonas Nasimzada
Department of Computer Science
University of Stuttgart
70569 Stuttgart
`st171793@stud.uni-stuttgart.de`

August 16, 2025

ABSTRACT

This study introduces ChessLLM, an chess engine leveraging Large Language Models (LLMs) and reinforcement learning to replicate human-like strategic understanding of chess. The research systematically compares custom Transformer architectures [1] and pre-trained LLMs (Llama 3.2-3B [2]), exploring their capabilities to internalize chess strategies through linguistic pattern recognition. ChessLLM employs a multi-stage training pipeline involving supervised fine-tuning on expert-level chess games and subsequent optimization via Proximal Policy Optimization (PPO) [3] and Group Relative Policy Optimization (GRPO) [4]. By integrating context-aware training methods that use historical move sequences in Forsyth-Edwards Notation (FEN) [5] and structured reasoning prompts, ChessLLM enhances interpretability and strategic accuracy.

Experimental evaluations against baseline engines, including recursive Minimax with Alpha-Beta pruning and Stockfish, demonstrate ChessLLM’s effectiveness. Remarkably, the model achieved a 98% win rate against Stockfish at skill level 0, underscoring its ability to balance tactical precision with strategic foresight.

All code, trained models, and datasets are available at <https://github.com/JonasNasimzada/ChessLLM>.

1 Introduction

Chess has long served as a benchmark for artificial intelligence research, representing a complex domain that requires strategic thinking, pattern recognition, and long-term planning. Traditional chess engines like Stockfish [6] and AlphaZero [7] have achieved superhuman performance through tree search algorithms and deep reinforcement learning respectively. However, these approaches fundamentally differ from human chess cognition, which relies heavily on pattern recognition, strategic understanding, and intuitive reasoning rather than exhaustive search.

The emergence of Large Language Models (LLMs) has revolutionized natural language processing and demonstrated remarkable capabilities in reasoning, planning, and problem-solving across diverse domains. This raises an intriguing question: can LLMs develop strategic chess understanding through language-based learning, similar to how humans learn chess through studying annotated games and strategic concepts?

Recent advances in LLM capabilities suggest that these models can internalize complex patterns and relationships when trained on appropriately structured data [8]. Chess notation provides a natural bridge between the symbolic representation of chess moves and the linguistic patterns that LLMs excel at processing. By representing chess positions in Forsyth-Edwards Notation (FEN) and moves in Universal Chess Interface (UCI) format, we can leverage the pattern recognition capabilities of LLMs for strategic game play.

This work presents ChessLLM, a comprehensive approach to training language models for chess that combines supervised fine-tuning with reinforcement learning. Our methodology explores whether LLMs can learn strategic chess play through language understanding rather than traditional tree search, potentially offering insights into how AI systems can develop strategic reasoning capabilities.

Contributions: Our work makes the following key contributions:

1. **Novel Architecture Comparison:** We systematically compare different neural architectures for chess play, including custom Transformers and pre-trained LLMs (Llama 3.2-3B), evaluating their effectiveness in learning chess strategy.
2. **Multi-Stage Training Pipeline:** We develop a comprehensive training methodology that combines supervised fine-tuning on expert games with reinforcement learning optimization using both Proximal Policy Optimization (PPO) and Group Relative Policy Optimization (GRPO).
3. **Context-Aware Chess Understanding:** We implement a context-aware approach that provides models with historical move sequences in FEN notation, enabling better strategic understanding and position evaluation.
4. **Comprehensive Evaluation Framework:** We establish a robust evaluation methodology by testing our models against classical chess engines (Minimax with Alpha-Beta pruning) and Stockfish.
5. **Chain-of-Thought Integration:** We incorporate structured reasoning prompts that guide the model through systematic position evaluation, including material count, pawn structure, king safety, and tactical considerations.

Our experimental results demonstrate that LLMs can indeed learn strategic chess play, while exhibiting more human-like decision-making patterns. The ChessLLM approach opens new avenues for developing AI systems that combine symbolic reasoning with language understanding.

2 Preliminary

2.1 Chess Fundamentals and Notation

Chess is a two-player strategy game played on an 8×8 board with 64 squares. Each player controls 16 pieces: one king, one queen, two rooks, two bishops, two knights, and eight pawns. The objective is to checkmate the opponent’s king, placing it under attack with no legal escape moves.

Forsyth-Edwards Notation (FEN): FEN is a standard notation for describing a specific chess position. A FEN string consists of six space-separated fields:

1. Piece placement (from rank 8 to rank 1, file a to h)
2. Active color (w for white, b for black)
3. Castling availability (KQkq for white/black kingside/queenside)
4. En passant target square
5. Halfmove clock (moves since last pawn move or capture)
6. Fullmove number

Example: `rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1`

Universal Chess Interface (UCI): UCI is a protocol for communicating with chess engines, using algebraic notation for moves. Moves are represented as four characters indicating the source and destination squares (e.g., `e2e4` for moving a piece from e2 to e4), with an optional fifth character for pawn promotion.

2.2 Large Language Models and Chess

Large Language Models have demonstrated remarkable capabilities in understanding and generating human-like text across various domains [9]. The application of LLMs to chess presents unique challenges and opportunities:

Challenges:

- Chess requires precise logical reasoning and adherence to complex rules
- The combinatorial nature of chess positions demands systematic evaluation
- Traditional chess engines rely on tree search, fundamentally different from language modeling

Opportunities:

- Chess notation provides a structured symbolic language suitable for LLM processing

- Annotated chess games contain rich strategic commentary that LLMs can learn from
- Pattern recognition in chess positions aligns with LLMs’ strength in identifying linguistic patterns

2.3 Model Architectures

Our research explores mainly two distinct neural architectures for chess play:

Custom Transformer: A purpose-built transformer architecture optimized for chess:

- 6-layer TransformerEncoder with 8 attention heads
- 960-dimensional model size matching our chess representation
- Designed specifically for processing board-move combinations

Pre-trained Language Model: Llama 3.2-3B-Instruct, a state-of-the-art language model with:

- 3 billion parameters pre-trained on diverse text corpora
- Instruction-tuning for following complex prompts
- Chat template support for structured conversation formats

2.4 Training Methodologies

Supervised Fine-Tuning (SFT) [10]: We begin with supervised learning on a curated dataset of expert chess games. The LumbrasGigaBase dataset provides 2.5 million annotated games with expert commentary and detailed move analysis. We convert PGN format games to FEN-UCI pairs, creating over 200 million training examples.

Parameter-Efficient Fine-Tuning (PEFT) [11]: To optimize computational resources, we employ LoRA (Low-Rank Adaptation) with rank 16, targeting specific transformer modules (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj). This approach freezes most pre-trained parameters while introducing trainable adapter layers.

Reinforcement Learning [12]: Reinforcement Learning involves learning how to choose actions based on different situations to maximize a numerical reward. Unlike supervised approaches, the learner isn’t explicitly instructed on what actions to take but instead must discover the best actions through experimentation. In the most intriguing and complex scenarios, actions can influence not only immediate rewards but also future situations, consequently affecting all later rewards. Two fundamental aspects distinguish reinforcement learning from other methods: trial-and-error exploration and delayed rewards.

Following supervised fine-tuning, we apply reinforcement learning to improve strategic play:

- **Proximal Policy Optimization (PPO)**: Used for custom transformer models, with a trust region approach to prevent large policy updates
- **Group Relative Policy Optimization (GRPO)**: Applied to pre-trained LLMs, generating multiple candidate moves per position and optimizing relative performance

3 Methodology

In order to train and evaluate ChessLLM as a language-based chess engine, we developed a comprehensive, multi-stage methodology that spans data preparation, prompt engineering, model design, supervised fine-tuning, and reinforcement-learning optimization. First, we curate and preprocess a high-quality corpus of expert-annotated games, converting raw PGN files into paired FEN-UCI examples to form the basis of both imitation and policy learning. Next, we design conversation-style, chain-of-thought prompts that feed the model not only the current board state but also a configurable window of preceding moves in Forsyth–Edwards Notation, thereby structuring each training instance as an intuitive question-answer dialogue. We then explore two distinct neural architectures, a purpose-built Transformer encoder and the instruction-tuned Llama 3.2-3B model, applying parameter-efficient fine-tuning (LoRA) to adapt large pre-trained weights without prohibitive compute costs. Having established a strong imitation-learning foundation through supervised fine-tuning, we further refine policy performance by integrating reinforcement-learning algorithms: Proximal Policy Optimization (PPO) for custom Transformers and Group Relative Policy Optimization (GRPO) for the LLM. Finally, we evaluate our trained agents against classical baselines, recursive Minimax with Alpha–Beta pruning and Stockfish, to quantify improvements in win rate, game length, and strategic robustness. The following subsections describe each component of this pipeline in detail.

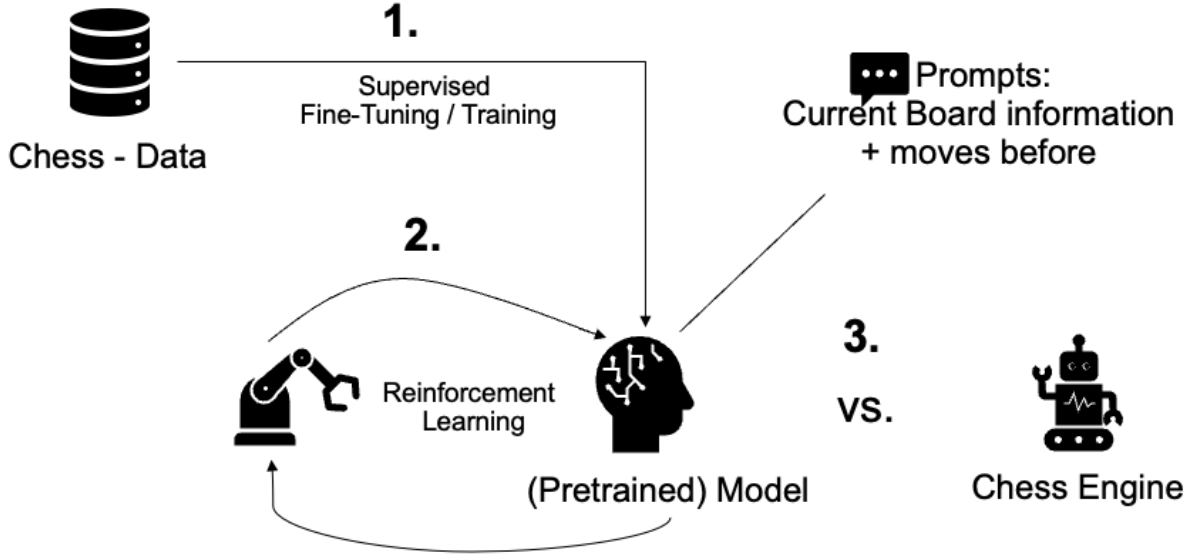


Figure 1: Training Pipeline of the ChessLLM

3.1 Dataset curation

To train our ChessLLM, we employed LumbrasGigaBase [13], a comprehensive dataset specifically curated for chess-related processing tasks. LumbrasGigaBase comprises an extensive collection of chess games, analyses, commentaries, and educational resources sourced from reputable chess databases, forums, and instructional websites. The dataset includes approximately 2.5 million annotated chess games featuring expert commentary, player annotations, and detailed move-by-move analyses. Moreover, LumbrasGigaBase integrates substantial metadata, such as player ratings, event details, and timestamps.

Initially, the textual data was stored in the traditional Portable Game Notation (PGN) format. PGN is a standard plain-text format for representing chess games in a human- and machine-readable way. For our specific use case, we converted each game’s data from PGN to UCI (Universal Chess Interface) notation for moves and FEN (Forsyth–Edwards Notation) for the corresponding board states, resulting in over 200 million chess move data points. FEN is a single-line, plain-text format for describing a specific chess position. UCI is a lightweight, text-based protocol that standardizes how a chess engine communicate with a fixed set of commands. Due to limited computational resources, we selected a subset comprising 1 million chess moves for pretraining our models.

3.2 Prompt-Generation

To effectively feed data points into our model, we structured them as conversational prompts enhanced with chain-of-thought (CoT) reasoning [14]. Alongside the current chessboard state, we included previously executed steps represented in Forsyth–Edwards Notation (FEN), as illustrated in Figure 1. These conversational prompts simulate natural dialogue, guiding the language model through systematic reasoning processes. This approach significantly improves interpretability and enhances the model’s strategic decision-making capabilities.

Conversational prompts adopt a question-answer dialogue format that mirrors human interactions, facilitating intuitive and contextually accurate responses from the language model (LLM). Specifically, for our ChessLLM, these prompts explicitly contextualize the chessboard situation provided in FEN notation and clearly request the optimal next move expressed in Universal Chess Interface (UCI) notation. This structured yet natural interaction enables the model to more effectively grasp the complexities of each chess position and formulate precise, context-aware moves.

CoT prompting encourages the model to explicitly outline its reasoning steps before reaching a final decision [14]. This strategy prompts the LLM to systematically process and articulate intermediate logical steps, resulting in more robust and explainable decision-making. By integrating conversational prompts and CoT reasoning into our ChessLLM, we hope to significantly improve both performance and interpretability, ultimately creating a more effective and reliable chess-playing language model.

To ensure the model returns moves derived from historical context, we explicitly include past 15 moves, thereby enhancing its contextual understanding.

By implementing this contextual framework-feeding the LLM not only the current position but also the full sequence of preceding moves in FEN notation, the model gains a comprehensive view of the game’s progression. This enriched context enables the LLM to recognize long-term strategic patterns, assess accumulated positional advantages or disadvantages, and avoid tactical pitfalls. As a result, the ChessLLM can generate moves that are more coherent with prior strategy, demonstrate improved foresight, and adapt dynamically to evolving board states, ultimately leading to stronger, more human-like play.

```
System (role =      system      ):

You are the worlds strongest chess engine. You will be given the full
move-history in FEN notation followed by the current position in FEN. Your
task is to think through the position step by step-evaluating piece placement,
pawn structure, king safety, candidate moves and tactical motifs-and then
output exactly one best move in UCI format.

Step-by-step guide:
  1. Material count and piece activity
  2. Pawn structure and central control
  3. King safety for both sides
  4. Candidate moves (e.g. developing, challenging the bishop, castling)
  5. Tactical considerations (pins, forks, discovered attacks)
  6. Long-term strategic plans

After reasoning, output only the best move in UCI format.

User (role =      user      ):

Move history (in FEN):
1.  rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq - 0 1
2.  rnbqkbnr/pp1ppppp/2p5/8/4P3/8/PPPP1PPP/RNBQKBNR w KQkq - 0 2
...
14. rn1qkbb1r/pp3ppp/2p2p2/8/3P2b1/5N2/PPP2PPP/R1BQKB1R w KQkq - 1 7
15. rn1qkbb1r/pp3ppp/2p2p2/8/3P2b1/5N2/PPP1BPPP/R1BQK2R b KQkq - 2 7

Current position (FEN):
  rn1qkbb1r/pp3ppp/5p2/2p5/3P2b1/5N2/PPP1BPPP/R1BQK2R w KQkq - 0 8

What is the next best move in UCI format?
```

Listing 1: Chess-Engine Prompt

3.3 Neural Networks

For our experiments, we employed two neural network models: (1) a Transformer architecture (see Figure 2) and (2) the autoregressive language model Llama 3.2 with 3 billion parameters, which has been pretrained and instruction-tuned. Llama 3.2 leverages an optimized Transformer backbone trained via causal language modeling, where each token prediction is conditioned on all preceding tokens. By representing chess moves as a text sequence, we enable imitation learning through supervised fine-tuning on our game dataset followed by reinforcement learning, allowing the model to refine its policy in a purely text-based framework.

3.4 Supervised Fine-Tuning

Fine-tuning represents a critical and highly effective step following the pre-training of foundation LLMs. Pre-trained LLMs have demonstrated substantial progress in language understanding and generation. Yet, these models are typically trained on extensive unsupervised text data and may not inherently be optimized for specific tasks. Fine-tuning addresses this limitation by leveraging the general linguistic knowledge obtained during pre-training and adapting it through supervised learning to suit a particular target task. When fine-tuning a pre-trained LLM on a task-specific labeled dataset, the model can achieve remarkable results using significantly fewer data and computational resources compared to training entirely new models from scratch.

Transformer

- 1.Input Layer:** 960-d feature vector
- 2.Head:** Linear transform to 128-d
- 3.Move:** 128-d→128-d linear block
- 4.Transformer Encoder:** 6 layers + 8 attention heads
each
- 5.Linear:** 128-d
- 6.Linear:** 128-d→1-d

Figure 2: Transformer Architecture

During supervised fine-tuning, the weights of the pre-trained LLM are adjusted based on gradients computed from a task-specific loss function, which measures the discrepancy between the model’s predictions and the actual ground-truth labels. This supervised process allows the model to capture nuanced patterns unique to the labeled dataset, refining its parameters to become adept at the particular task.

To enhance resource efficiency, parameter-efficient fine-tuning (PEFT) techniques are employed. PEFT achieves efficiency by freezing the majority of the pretrained model’s parameters, introducing only a small set of trainable parameters, known as adapters, in the final layers. These adapters are specialized for predetermined downstream tasks, enabling the fine-tuned models to retain their broad pre-trained knowledge while efficiently adapting to specific tasks. As a result, PEFT substantially reduces both computational and storage demands while maintaining strong task-specific performance.

We supervised fine-tuned with the Dataset of 1 million datapoints. As an optimizer we used AdamW with a learning rate of $2e-4$ and a weight decay of 0.01. We also used a linear learning rate scheduler with warmup steps of 5. Finally we performed totally 3 epoch of training, as seen in Table 1. The PEFT configuration is seen in Figure 2

Parameter	Value
per_device_train_batch_size	2
gradient_accumulation_steps	4
warmup_steps	5
num_train_epochs	3
learning_rate	$2e-4$
logging_steps	1
optim	"adamw_8bit"
weight_decay	0.01
lr_scheduler_type	"linear"
seed	3407
report_to	"wandb"
dataset_text_field	"text"
max_seq_length	2048
dataset_num_proc	2
packing	False
ddp_find_unused_parameters	False

Table 1: Configuration parameters for Supervised Fine-Tuning.

3.5 Reinforcement Learning

To further improve our models as chess engines, we incorporated reinforcement learning. For the Transformer, we used Proximal Policy Optimization (PPO) with a terminal reward of +1 for wins, −1 for losses, and 0 for draws (see Figure

Parameter	Value
r	16
Target Modules	["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]
LoRA Alpha	16
LoRA Dropout	0
Bias	none
Use Gradient Checkpointing	unsloth
Use RsLoRa	False
LoFTQ Config	None

Table 2: Configuration Parameters for PEFT

3). For the language model, we applied Group Relative Policy Optimization (GRPO), a recent algorithm introduced by DeepSeek. Our reward structure comprised four components:

- **Game outcome:** +1 for a win, −1 for a loss, and 0 for a draw.
- **Material advantage:** rewards for capturing pieces and penalties for losing them, encouraging the model to prioritize material gain and win in as few moves as possible.
- **Move validity:** a positive reward for producing valid UCI moves, ensuring all outputs conform to the chess protocol.
- **Expert imitation:** a reward when the model’s move matches one from our master-level game dataset in the same board situation, supporting learning from grandmaster play.

	<u>Transformer</u>	<u>LLM</u>
<u>Algorithm</u>	Proximal Policy Optimization (PPO)	Group Relative Policy Optimization (GRPO)
<u>Reward Structure</u>	• Terminal rewards of +1 for Wins, −1 for Losses & Draw	• Win, Loss, Draw game • Winning/Losing piece • Valid response • Response from Dataset

Figure 3: Reinforcement Learning settings for the models

We used reinforcement learning with the Dataset of 130.000 datapoints. As a optimizer we used AdamW with a learning rate of $5e-6$ and a weight decay of 0.01. We also used a cosine learning rate scheduler. Finally we performed totally 3 epoch of training, as seen in Table 3. The PEFT configuration are the same as for the supervised fine-tuning in Table 2.

3.5.1 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a family of on-policy, policy-gradient methods designed to optimize the balance between ease of implementation, sample efficiency, and reliable convergence. In contrast to classical policy-gradient algorithms, like Trust Region Policy Optimization, (TRPO), PPO replaces the hard trust-region constraint with a simple clipping mechanism that prevents any individual policy update from moving too far from the previous policy.

Figure 4 illustrates the data flow and learning architecture underlying Proximal Policy Optimization (PPO). Starting from an input context q , the Policy Model generates an output action o , which is then evaluated by the Reference Model and the Reward Model. The Reference Model computes the log-probability of o under the previous policy parameters θ_{old} , from which a Kullback–Leibler divergence term $\text{KL}[\pi_{\theta}(\cdot | q) \parallel \pi_{\theta_{old}}(\cdot | q)]$ is derived. Simultaneously, the Reward Model assigns a reward r_{raw} to the same action–context pair. These two signals are combined to form an effective reward

$$r = r_{raw} - \beta \text{KL}[\pi_{\theta}(\cdot | q) \parallel \pi_{ref}(\cdot | q)].$$

Parameter	Value
learning_rate	5e-6
weight_decay	0.1
warmup_ratio	0.1
lr_scheduler_type	cosine
optim	adamw_8bit
logging_steps	1
per_device_train_batch_size	8
gradient_accumulation_steps	4
num_generations	4
max_prompt_length	1200
max_completion_length	840
num_train_epochs	3
max_grad_norm	1.0

Table 3: Configuration Parameters for GRPO

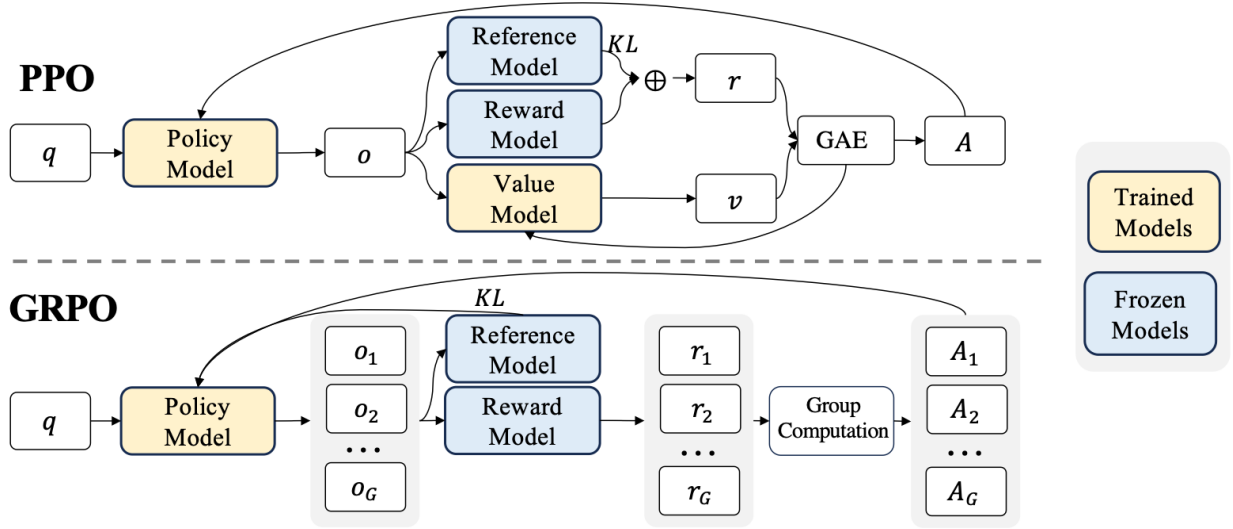


Figure 4: Architectur of the PPO & GRPO algorithm

thereby softly penalizing excessive policy shifts away from the reference distribution. In parallel, the Value Model produces a state-value estimate $v \approx V^\pi(q)$ for the current context. Both the penalized reward stream r and the value prediction v feed into a Generalized Advantage Estimation (GAE) block, which computes a smoothed advantage estimate $\hat{A}_t$. Finally, these advantage estimates drive simultaneous updates of the trainable components. The Policy Model parameters θ are adjusted by ascending the clipped surrogate objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right], \text{ where } r_t(\theta) = \pi_\theta(o_t | q_t) / \pi_{\theta_{\text{old}}}(o_t | q_t)$$

By freezing the Reference and Reward Models and solely training the Policy and Value Models, this architecture ensures that external evaluators remain stable while the policy incrementally improves within a trust-region defined by the clipping mechanism.

3.5.2 Group Relative Policy Optimization (GRPO)

Group Relative Policy Optimization (GRPO), which extends the single-sample updates of PPO to a set of parallel rollouts before computing learning signals. Given a shared input context q , the Policy Model emits a batch of actions

$$o_1, o_2, \dots, o_G \sim \pi_\theta(\cdot | q).$$

as seen in Figure 4.

Each action o_i is then evaluated by two frozen networks: a Reference Model that returns the log-probability under the previous policy $\log \pi_{\theta_{\text{old}}}(o_i | q)$, and a Reward Model that produces a raw scalar reward $r_{\text{raw},i}$. Following PPO’s soft trust-region principle, we combine these signals to form a penalized reward

$$r_i = r_{\text{raw},i} - \beta \text{KL}[\pi_{\theta}(\cdot | q) \parallel \pi_{\theta_{\text{old}}}(\cdot | q)] ,$$

ensuring the new policy does not drift excessively from its reference. Finally, GRPO performs a policy update by maximizing a group-aggregated clipped surrogate objective:

$$L^{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \mathbb{E} \left[\min(r_t^i(\theta) \hat{A}_i, \text{clip}(r_t^i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i) \right], \text{ where } r_t^i(\theta) = \pi_{\theta}(o_i | q) / \pi_{\theta_{\text{old}}}(o_i | q)$$

Group Relative Policy Optimization (GRPO) improves on vanilla PPO by Lower-Variance Advantage Estimates, Emphasis on Relative Performance and Robustness to Reward Scale and Noise

3.6 Chess Engine

In order to evaluate ChessLLM, a series of matches were conducted against two distinct chess engines: (a) Recursive Minimax with Alpha–Beta and (b) Stockfish, with a skill level of 0.

3.6.1 Recursive Minimax with Alpha–Beta

The opponent engine uses a depth-limited Minimax search with Alpha–Beta pruning [15] to select its move. At each node, it alternates between maximizing (the agent’s turn) and minimizing (the opponent’s turn) the evaluation score, which is computed by a simple material-balance function. Alpha–Beta bounds are used to cut off branches that cannot influence the final decision. The Minimax algorithm is a recursive decision rule used in two-player, zero-sum games to determine the optimal move by exhaustively exploring the game tree to a fixed depth. At each node, the “maximizing” player selects the child with the highest evaluation score, while the “minimizing” opponent selects the child with the lowest score. Leaf nodes are scored by a heuristic evaluation function (e.g., material balance in chess), and these scores propagate upward: maximizing nodes pick the maximum of their children’s values, minimizing nodes pick the minimum. Alpha–Beta pruning is an enhancement of Minimax that eliminates branches of the search tree which cannot possibly influence the final decision, thereby reducing the number of evaluated nodes without affecting the result. Two parameters, α (the best already explored option for the maximizer) and β (the best for the minimizer), are maintained during the search. Whenever $\alpha \geq \beta$ at a node, further exploration of that node’s children is unnecessary (a cutoff), because the current player can force a better outcome elsewhere. This pruning yields the same Minimax decision with substantially less computation.

The search depth was set to 3. The skill level of this chess engine corresponds to approximately 800-1000 Elo, classifying it as a beginner-level player.

3.6.2 Stockfish

Stockfish is an open-source chess engine that consistently ranks among the world’s strongest. Written primarily in C++ with hand-optimized assembly for critical routines, it employs an iterative-deepening minimax search enhanced with alpha–beta pruning, together with forward-pruning heuristics such as null-move pruning, late-move reductions, and aspiration windows to efficiently reduce the search tree.

For evaluation, we used Stockfish 16 configured at skill level 0, a thinking time of 100 ms, and a search depth of 1. This setup corresponds to an Elo rating of approximately 1300-1500, characteristic of an amateur-level player.

4 Evaluation

We conducted evaluation batches consisting of 100 games each, in which our chess models played against (a) a recursive minimax algorithm with alpha-beta pruning and (b) Stockfish at skill level 0. We selected the batch with the best overall performance and recorded the number of wins, losses, and draws, the average number of moves per game, and the overall win rate. Table 4 summarizes these results.

Against the Minimax engine, the plain Transformer, trained only on a “win/draw/lose” reward, secured 45 wins and no losses, but half of its games ended in draws, and play extended to an average of 147 half-moves. This result underscores

Table 4: Evaluation results of our chess models against baseline engines are summarized below. The model labeled SF represents the purely supervised fine-tuned model. For all reinforcement learning models, we additionally incorporated Move Validity and Expert Imitation rewards to ensure the validity of processed results.

Model	Reward	Opponent	Wins	Losses	Draws	Avg. Moves	Win Rate (%)
Plain Transformer	Win / Draw / Lose	Minimax	45	0	55	147	45
Pretrained Transformer	Win / Draw / Lose	Minimax	50	0	50	99	50
SF - Llama 3.2 (3B)	None	Minimax	55	1	44	88	55
Llama 3.2 (3B)	Win / Draw / Lose	Minimax	64	1	35	139	64
Llama 3.2 (3B)	Win / Draw / Lose	Stockfish	95	5	0	77	95
Llama 3.2 (3B)	Material Advantage	Minimax	67	0	33	83	67
Llama 3.2 (3B)	Material Advantage	Stockfish	95	5	0	56	95
Llama 3.2 (3B)	Win / Draw / Lose & Material Advantage	Minimax	64	0	36	105	64
Llama 3.2 (3B)	Win / Draw / Lose & Material Advantage	Stockfish	98	1	1	71	98

the model’s ability to avoid defeat but also its hesitancy when guided solely by outcome signals. Introducing large-scale supervised pretraining on a LLM (SF-Llama 3.2) raised the win rate to 50 % and cut game length by one-third (99 moves on average), illustrating the value of expert data in shaping opening and middlegame understanding .

When we applied reinforcement learning with a reward for material advantage, the model’s style shifted decisively: it achieved a 67 % win rate against Minimax with zero losses and reduced the average game to 83 moves. By incentivizing piece gains, the agent learned more decisive tactics, trading precision for aggression. Conversely, the combined “win/draw/lose + material” reward yielded a balanced profile, 64 % wins, zero losses, and an average of 105 moves, demonstrating that blending outcome, level and heuristic signals can harmonize exploratory play with tactical efficiency.

The contrasts become even starker against Stockfish. The outcome-only RL agent recorded a 95 % win rate (95 wins, 5 losses) in 77 moves on average, while the material-only agent matched this win rate but slashed game length to 56 moves, emphasizing how material incentives accelerate victories even against a stronger adversary. Ultimately, the combined-reward model attained the apex of performance, 98 wins, 1 loss, and 1 draw (98 % win rate) in a concise 71 moves, highlighting that synergistic reward shaping produces both robust and efficient play against traditional engines.

These findings demonstrate that reinforcement-learning reward design critically influences LLM-based chess performance. Outcome rewards drive raw success rates, material rewards foster rapid, tactical victories, and their combination yields a versatile agent capable of overwhelming both heuristic and search-based opponents with high precision and speed.

4.1 Limitations

Our results might differ from other studies [16] [17] due to several factors. Firstly, the prompts we used may have been overly long, potentially causing the model difficulty in comprehending the historical board state represented in FEN notation. A possible improvement would be shortening the prompts or employing UCI notation, or even omitting the history altogether. Secondly, the comparatively limited size of our training dataset could be another cause, as related studies demonstrating better performance typically utilized larger datasets with shorter prompts. Finally, integrating reinforcement learning by evaluating moves using Stockfish, rather than solely relying on dataset-based moves with GRPO, could mitigate the risk of the model inadvertently learning incorrect moves from the dataset.

5 Conclusions

In this paper, we introduced ChessLLM, an innovative chess engine leveraging Large Language Models (LLMs) combined with reinforcement learning methodologies to emulate human-like strategic understanding of chess. By systematically comparing custom Transformer architectures and the state-of-the-art pre-trained Llama 3.2-3B model, we demonstrated that LLMs can effectively internalize complex chess strategies through language-based training approaches.

Our multi-stage training pipeline, consisting of supervised fine-tuning on expert-level games and subsequent optimization through Proximal Policy Optimization (PPO) and Group Relative Policy Optimization (GRPO), significantly improved the strategic decision-making capabilities of our models. Additionally, employing a context-aware approach and structured reasoning via chain-of-thought prompts enhanced both the interpretability and effectiveness of our model.

Experimental results indicate that reinforcement learning rewards critically shape the performance and strategic tendencies of LLM-based chess engines. The ChessLLM model notably achieved a 98% win rate against Stockfish at skill level 0, underscoring its effectiveness in balancing tactical precision with strategic foresight.

However, several limitations, including the prompt length and dataset scale, suggest potential avenues for future improvement. Future research should explore shorter, more efficient prompting strategies, larger and more diverse datasets, and further integration of reinforcement learning evaluation using advanced chess engines to enhance overall strategic accuracy.

Ultimately, ChessLLM represents a meaningful step towards AI systems that not only excel in complex strategic tasks but also closely resemble human cognitive processes in strategic reasoning and decision-making.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762> (cit. on p. 1).
- [2] A. Grattafiori et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs.AI]. URL: <https://arxiv.org/abs/2407.21783> (cit. on p. 1).
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347> (cit. on p. 1).
- [4] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, D. Guo. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. 2024. arXiv: 2402.03300 [cs.CL]. URL: <https://arxiv.org/abs/2402.03300> (cit. on p. 1).
- [5] *FEN (Forsyth-Edwards Notation) - Chess Terms* — [chess.com](https://www.chess.com/terms/fen-chess). <https://www.chess.com/terms/fen-chess>. [Accessed 19-07-2025] (cit. on p. 1).
- [6] *Stockfish* — stockfishchess.org. <https://stockfishchess.org>. [Accessed 19-07-2025] (cit. on p. 1).
- [7] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, D. Hassabis. *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. 2017. arXiv: 1712.01815 [cs.AI]. URL: <https://arxiv.org/abs/1712.01815> (cit. on p. 1).
- [8] F. Wu, V. P. Dwivedi, J. Leskovec. *Large Language Models are Good Relational Learners*. 2025. arXiv: 2506.05725 [cs.CL]. URL: <https://arxiv.org/abs/2506.05725> (cit. on p. 1).
- [9] N. Karanikolas, E. Manga, N. Samaridi, E. Tousidou, M. Vassilakopoulos. “Large Language Models versus Natural Language Understanding and Generation”. In: *Proceedings of the 27th Pan-Hellenic Conference on Progress in Computing and Informatics*. PCI ’23. Lamia, Greece: Association for Computing Machinery, 2024, pp. 278–290. ISBN: 9798400716263. DOI: 10.1145/3635059.3635104. URL: <https://doi.org/10.1145/3635059.3635104> (cit. on p. 2).
- [10] A. Pareja, N. S. Nayak, H. Wang, K. Killamsetty, S. Sudalairaj, W. Zhao, S. Han, A. Bhandwaldar, G. Xu, K. Xu, L. Han, L. Inglis, A. Srivastava. *Unveiling the Secret Recipe: A Guide For Supervised Fine-Tuning Small LLMs*. 2024. arXiv: 2412.13337 [cs.LG]. URL: <https://arxiv.org/abs/2412.13337> (cit. on p. 3).
- [11] L. Xu, H. Xie, S.-Z. J. Qin, X. Tao, F. L. Wang. *Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment*. 2023. arXiv: 2312.12148 [cs.CL]. URL: <https://arxiv.org/abs/2312.12148> (cit. on p. 3).

- [12] M. Ghasemi, D. Ebrahimi. *Introduction to Reinforcement Learning*. 2024. arXiv: 2408.07712 [cs.AI]. URL: <https://arxiv.org/abs/2408.07712> (cit. on p. 3).
- [13] *Free chess game database for Scid and PGN — lumbrasgigabase.com*. <https://lumbrasgigabase.com/en/>. [Accessed 19-07-2025] (cit. on p. 4).
- [14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, D. Zhou. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. arXiv: 2201.11903 [cs.CL]. URL: <https://arxiv.org/abs/2201.11903> (cit. on p. 4).
- [15] S. Shevtekar, M. Malpe, M. Bhaila. “Analysis of Game Tree Search Algorithms Using Minimax Algorithm and Alpha-Beta Pruning”. In: *International Journal of Scientific Research in Computer Science, Engineering and Information Technology* (Nov. 2022), pp. 328–333. DOI: 10.32628/CSEIT1228644 (cit. on p. 9).
- [16] Y. Zhang, X. Han, H. Li, K. Chen, S. Lin. *Complete Chess Games Enable LLM Become A Chess Master*. 2025. arXiv: 2501.17186 [cs.AI]. URL: <https://arxiv.org/abs/2501.17186> (cit. on p. 10).
- [17] X. Feng, Y. Luo, Z. Wang, H. Tang, M. Yang, K. Shao, D. Mguni, Y. Du, J. Wang. *ChessGPT: Bridging Policy Learning and Language Modeling*. 2023. arXiv: 2306.09200 [cs.LG]. URL: <https://arxiv.org/abs/2306.09200> (cit. on p. 10).